1. Objective

The aim of this lab is to understand how frequent patterns in YouTube comments change when data arrives gradually instead of all at once. To simulate this real-world situation, the cleaned comment dataset from Lab 2 is divided into five parts ("chunks"). Each chunk is treated like a time segment, and frequent words (unigrams) and frequent co-occurring word pairs are mined from each chunk separately. After observing how patterns change from chunk to chunk, correlation analysis is used to check whether certain words rise and fall together across the five chunks. This helps identify relationships between themes in the discussion.

2. Tools and Libraries Used

This lab was implemented using Python 3.13.7. The python libraries used:

- **pandas**: for loading the CSV dataset and working with tables (DataFrames)
- **numpy**: for splitting the dataset into five chunks to simulate incremental arrival
- **ast**: for safely converting token strings into real Python lists when needed
- **collections.Counter**: for counting word frequencies (unigrams and pairs)
- **itertools.combinations**: for generating co-occurring word pairs inside each comment
- **matplotlib**: for plotting bar charts (top patterns per chunk) and line graphs (pattern trends over chunks)

3. Dataset Description

The dataset used in this lab is cleaned_comments.csv from Lab 2. The loaded dataset contains the columns:

['username', 'timestamp_text', 'comment_text', 'cleaned_tokens']

The most important column for this lab is cleaned_tokens, which contains a list of preprocessed words for each comment. Each comment is treated as a transaction of words (same idea as market-basket analysis, but here the "items" are words).

The dataset includes 739 total comments. To simulate incremental arrival, the data is split into 5 chunks. Each chunk represents "comments arriving over time" in a simplified way.

Important limitation:

The chunking is done by row order, not by real posted time. Even though the dataset has a timestamp_text column, this lab segmentation is based on row order. If the dataset rows are not truly chronological, the chunking acts only as a simulation of time, not the real posting sequence. This limitation is acknowledged in the interpretation.

4. Methodology / Procedure

4.1 Data Loading and Segmentation

The dataset was read using pandas. After reading, the code checked that the cleaned_tokens column exists. A safe conversion step was also applied because sometimes token lists are stored as strings inside CSV files. If the first row is a string, it converts all entries to real Python lists using ast.literal_eval.

```
df = pd.read_csv("cleaned_comments.csv")
if "cleaned_tokens" not in df.columns:
    raise ValueError("cleaned_tokens column not found.")
if isinstance(df['cleaned_tokens'].iloc[0], str):
    df['cleaned_tokens'] = df['cleaned_tokens'].apply(ast.literal_eval)
```

After loading, the script confirmed:

- Total comments: 739
- Data split into 5 chunks

Even though the lab manual suggests np.array_split(df, 5), my environment caused indexing issues when splitting the DataFrame directly. To keep each chunk as a valid DataFrame (as required by the lab), row indices were split first and then used to slice the DataFrame:

```
chunks = np.array_split(df.index, 5)
chunks = [df.loc[idx] for idx in chunks]
```

4.2 Pattern Mining per Chunk

Each chunk was analyzed separately. In each chunk:

1. The cleaned_tokens column was extracted as a list of transactions.
2. Top 10 unigrams were counted using Counter.
3. Top 10 co-occurring word pairs were counted using itertools.combinations().

The word pairs are unordered co-occurring pairs within the same comment, not adjacent phrase bigrams. This means the method captures association/themes (words appearing together) rather than exact word order or phrases.

```
transactions = chunk['cleaned_tokens'].tolist()
unigram_counts = Counter(token for row in transactions for token in row)
bigram_counts = Counter()
for row in transactions:
    bigram_counts.update(combinations(sorted(set(row)), 2))
```

For visualization, bar charts were plotted for each chunk (both unigrams and co-occurring pairs).

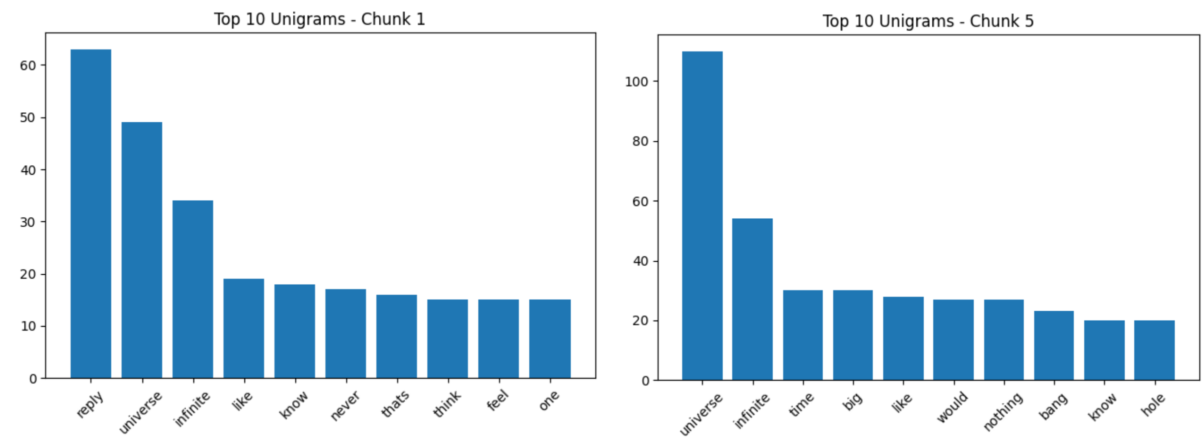

Figure 1: Chunk 1 & 5 Graph of Top 10 Unigrams

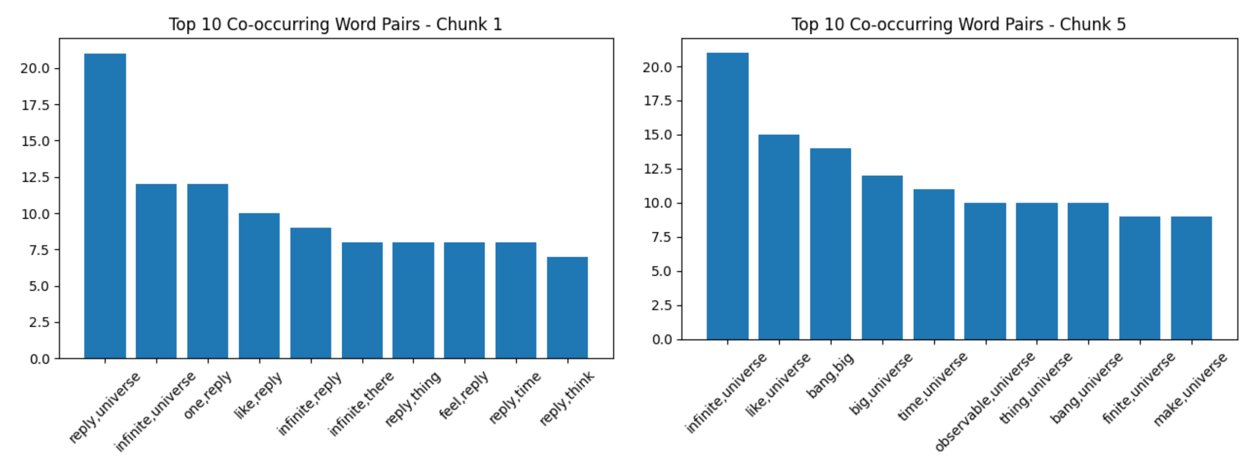

Figure 2: Chunk 1 & 5 Graph of Top 10 Co-occurring Word Pairs

4.3 Tracking Pattern Evolution

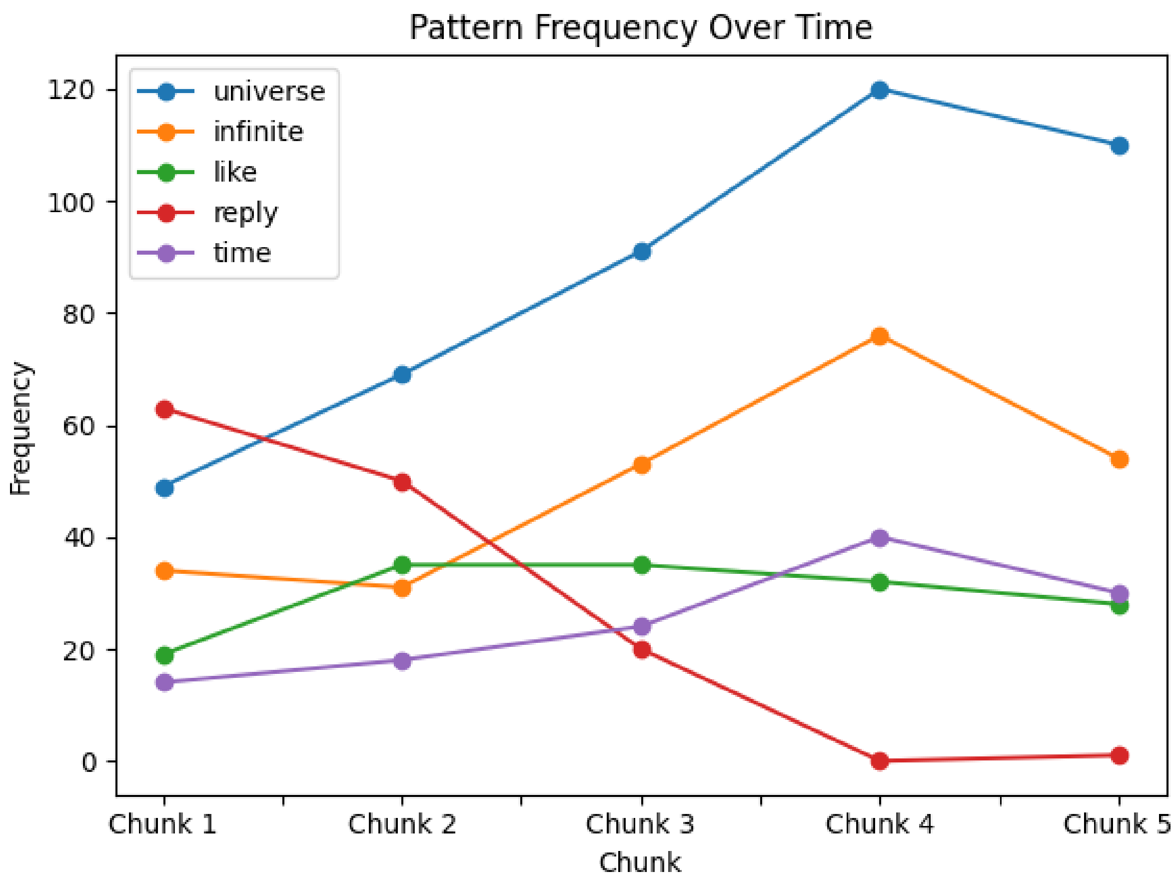

Figure 3: Pattern Frequency Over Time

After mining each chunk, the patterns were compared across chunks. The lab requires tracking how top patterns change, so the most frequent overall unigrams were selected and tracked over all five chunks.

The patterns selected for tracking were: ['universe', 'infinite', 'like', 'reply', 'time']

A frequency table was created with frequencies for each chunk. If a word did not appear in a chunk, its frequency was recorded as **0**.

4.4 Correlation Analysis

Correlation analysis was performed on the selected patterns to see whether they rise and fall together across the five chunks. A correlation matrix was created using: corr = freq_df.corr()

Interpretation rules used:

- Correlation close to **+1** → two patterns increase/decrease together strongly

- Correlation close to **-1** → one rises when the other falls (inverse relationship)
- Correlation near **0** → weak/no clear relationship

A line graph *Figure 3* was plotted for the selected patterns to visually compare their trends across chunks.

4.5 Reflection and Insight

Pattern: universe

- **Frequency change:** The word universe rises strongly across chunks (49 → 69 → 91 → 120) and then drop slightly in chunk 5 (110). Overall, it becomes more dominant over time.
- **Correlations:** It has a strong positive correlation with time (0.9609) and infinite (0.9044), but a very strong negative correlation with reply (-0.9880).
- **Possible reason:** As the discussion progresses, more comments focus directly on the main topic (the universe and related ideas like infinity and time), while reply-style conversational threads reduce, which explains why universe increases while reply decreases.

Pattern: infinite

- **Frequency change:** infinite stays similar early (34 → 31) and then rises in the middle and later chunks (53 → 76), before decreasing in chunk 5 (54). The peak is in chunk 4.
- **Correlations:** It is strongly positively correlated with time (0.9623) and universe (0.9044), and strongly negatively correlated with reply (-0.8856).
- **Possible reason:** Later chunks show more concept-heavy discussion (infinite universe, infinite time/space), suggesting that as viewers engage deeper with the topic, they use "infinite" more often, especially together with "universe" and "time."

Pattern: like

- **Frequency change:** like increases from chunk 1 to chunks 2–3 (19 → 35 → 35), then gradually declines (32 → 28). It is relatively stable but slightly decreasing later.
- **Correlations:** It has moderate correlation with universe (0.4731) and weak correlation with time (0.3689). It is mildly negatively correlated with reply (-0.4156).
- **Possible reason:** "Like" is often used in casual speech ("like…") or simple reactions, so it becomes common when comments are more conversational. As later chunks become more technical or topic-focused, "like" decreases slightly.

Pattern: reply

- **Frequency change:** reply starts very high (63), decreases in chunk 2 (50), drops sharply in chunk 3 (20), disappears in chunk 4 (0), and is almost absent in chunk 5 (1). This is the strongest change among the selected patterns.

- **Correlations:** It has extremely strong negative correlation with universe (-0.9880) and strong negative correlation with time (-0.9235), meaning reply decreases when those topic-words increase.
- **Possible reason (deeper insight):** Early chunks likely contain more thread-style interaction (people replying to each other), which produces many "reply" tokens. Later chunks may contain more standalone discussion comments focused on the video topic, reducing reply behavior and increasing topic words like "universe," "time," and "infinite."

Pattern: time

- **Frequency change:** time increases consistently from chunk 1 to chunk 4 (14 → 18 → 24 → 40) and then decreases in chunk 5 (30). This shows growing focus on time-related ideas mid-to-late.
- **Correlations:** It has very strong positive correlation with universe (0.9609) and infinite (0.9623), and strong negative correlation with reply (-0.9235).
- **Possible reason:** Discussions about the universe often involve time concepts (age of universe, infinite time, time and space). As the comment section shifts toward deeper explanations, "time" appears more frequently alongside "universe" and "infinite."

5. Code Implementation

5.1 Data Loading and Splitting

First, the dataset cleaned_comments.csv is loaded using pandas. After loading, the script verifies that the required column cleaned_tokens exists. Since CSV files sometimes store lists as strings, the code checks the first element type and converts the whole column into real Python lists using ast.literal_eval() if needed. This ensures that every row contains a usable list of tokens.

```
df = pd.read_csv("cleaned_comments.csv")
if "cleaned_tokens" not in df.columns:
    raise ValueError("cleaned_tokens column not found.")
if isinstance(df['cleaned_tokens'].iloc[0], str):
    df['cleaned_tokens'] = df['cleaned_tokens'].apply(ast.literal_eval)
```

After that, the dataset is split into five equal chunks to simulate incremental arrival of comments. The lab manual suggests using np.array_split(df, 5), but in this environment splitting the DataFrame directly caused indexing issues. To keep each chunk as a valid DataFrame (as required), the code splits row indices first and then slices the DataFrame using .loc.

```
chunks = np.array_split(df.index, 5)
chunks = [df.loc[idx] for idx in chunks
```

5.2 Pattern Counting (Unigrams and Co-Occurring Word Pairs)

For each chunk, the script extracts the list of token transactions:

```
transactions = chunk['cleaned_tokens'].tolist()
```

Unigram Counting: Unigrams are counted by flattening all tokens from all comments in the chunk and using Counter to calculate frequencies. The top 10 most frequent words are then printed.

```
unigram_counts = Counter(token for row in transactions for token in row)
top_unigrams = unigram_counts.most_common(10)
```

Co-Occurring Pair Counting: To mine word associations, the script counts unordered co-occurring pairs inside each comment using itertools.combinations(). Each comment's word list is converted to a unique set first to avoid duplicates inside the same comment, and then all 2-word combinations are counted. This method finds theme/association pairs (words appearing together) rather than adjacent phrase bigrams.

```
bigram_counts = Counter()
for row in transactions:
    unique_words = sorted(set(row))
    bigram_counts.update(combinations(unique_words, 2))
top_bigrams = bigram_counts.most_common(10)
```

5.3 Visualization (Matplotlib Plots)

Visualization is done for every chunk:

- A bar chart of top 10 unigrams
- A bar chart of top 10 co-occurring pairs

These plots make it easy to compare patterns across chunks visually.

Code snippet (unigram bar chart):

```
plt.figure()
plt.bar([w for w, _ in top_unigrams], [c for _, c in top_unigrams])
plt.title(f"Top 10 Unigrams - Chunk {i+1}")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

Code snippet (pair bar chart):

```
bigram_labels = [f"{a},{b}" for (a, b), _ in top_bigrams]
bigram_values = [c for _, c in top_bigrams]


plt.figure()
plt.bar(bigram_labels, bigram_values)
plt.title(f"Top 10 Co-occurring Word Pairs - Chunk {i+1}")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

A final visualization is also produced for the correlation section: a **line graph** showing how selected pattern frequencies change across chunks.

5.4 Correlation Computation

After all chunks are processed, the script selects the most frequent overall patterns and builds a frequency table across the five chunks. Missing values are recorded as **0**, as required.

Code snippet (frequency table):

```
total_unigram_counts = sum(all_unigram_counts, Counter())

selected_patterns = [word for word, _ in total_unigram_counts.most_common(5)]

freq_data = {}

for pattern in selected_patterns:

    freq_data[pattern] = [chunk_counts.get(pattern, 0) for chunk_counts in all_unigram_counts]


freq_df = pd.DataFrame(freq_data)

freq_df.index = [f"Chunk {i+1}" for i in range(5)]
```

Then, correlation is computed using pandas.corr():

Code snippet (correlation matrix):

```
corr = freq_df.corr()

print(corr)

Finally, the frequency trends are plotted to help interpret correlation visually:

freq_df.plot(marker='o')

plt.xlabel("Chunk")

plt.ylabel("Frequency")

plt.title("Pattern Frequency Over Time")

plt.tight_layout()

plt.show()
```

6. Results and Discussion

6.1 Top 10 Words (Unigrams) per Chunk

Chunk	Top 10 Unigrams (word, count)	Key Insight
1	reply (63), universe (49), infinite (34), like (19), know (18), never (17), thats (16), think (15), feel (15), one (15)	Chunk 1 is very **reply-heavy**, suggesting more conversation/thread behavior early, while the topic words *universe* and *infinite* are already strong.
2	universe (69), reply (50), like (35), infinite (31), video (25), time (18), would (17), edge (15), thing (13), even (13)	Topic discussion becomes stronger: *universe* increases and *time/edge* appear, showing more "explanation style" comments starting.
3	universe (91), infinite (53), like (35), light (25), time (24), big (24), video (23), space (23), one (22), reply (20)	Clear shift toward **physics/cosmology terms**: *space, light, big* appear, and *reply* drops a lot, meaning fewer thread-type comments.
4	universe (120), infinite (76), time (40), space (38), like (32), would (29), big (26), edge (24), exact (24), one (23)	Chunk 4 shows the **peak of topic depth**: *universe/infinite/time/space* all rise strongly, suggesting more detailed discussion and arguments.
5	universe (110), infinite (54), time (30), big (30), like (28), would (27), nothing (27), bang (23), know (20), hole (20)	New topic terms like *bang* and *hole* appear, suggesting the discussion expands into **Big Bang / black hole** type explanations in the last chunk.

Table 1: Top 10 Words (Unigrams)

Overall unigram insight:

The dataset becomes increasingly focused on the main topic (universe, infinite, time, space) as chunks progress. The word reply is strong early but fades later, which is an important pattern shift.

6.2 Top 10 Co-occurring Word Pairs per Chunk

Chunk	Top 10 Co-occurring Pairs ((w1, w2), count)	Key Insight
1	(reply, universe) 21; (infinite, universe) 12; (one, reply) 12; (like, reply) 10; (infinite, reply) 9; (infinite, there) 8; (reply, thing) 8; (feel, reply) 8; (reply, time) 8; (reply, think) 7	Chunk 1 associations are dominated by **reply + topic words**, showing early discussion is mixed with conversation threads.
2	(reply, universe) 18; (infinite, universe) 18; (universe, would) 12; (like, universe) 12; (reply, video) 10; (universe, video) 10; (like, reply) 9; (edge, universe) 8; (finite, universe) 8; (edge, reply) 7	Strong rise of **(infinite, universe)** and appearance of **finite/edge**, showing debate about the universe being finite/infinite.
3	(infinite, universe) 26; (big, universe) 15; (time, universe) 14; (one, universe) 10; (finite, universe) 9; (infinite, time) 9; (make, universe) 9; (like, universe) 9; (like, would) 9; (think, universe) 9	Discussion shifts from "reply" to **concept clusters** like *big/universe/time*, showing more explanatory arguments.
4	(infinite, universe) 35; (space, universe) 16; (time, universe) 16; (finite, universe) 13; (edge, universe) 13; (like, universe) 13; (infinite, space) 12; (one, universe) 11; (mean, universe) 11; (universe, would) 11	Peak association is **(infinite, universe)** (35). Also *space/time/universe* appear strongly, indicating deep topic focus.
5	(infinite, universe) 21; (like, universe) 15; (bang, big) 14; (big, universe) 12; (time, universe) 11; (observable, universe) 10; (thing, universe) 10; (bang, universe) 10; (finite, universe) 9; (make, universe) 9	New associations appear: **Big Bang pairs** (*bang, big* and *bang, universe*) and "observable universe," indicating topic expansion late.

Table 2: Top 10 Co-occurring Word Pairs

Overall pairs insight:

Across chunks 3–5, the pairs become less "reply-centered" and more concept-centered. The consistently strongest pair is (infinite, universe), showing the central theme is about whether/how the universe is infinite.

6.3 Pattern Evolution

Selected patterns tracked: **universe, infinite, like, reply, time**

Chunk	universe	infinite	like	reply	time
1	49	34	19	63	14
2	69	31	35	50	18
3	91	53	35	20	24
4	120	76	32	0	40
5	110	54	28	1	30

Table 3: Pattern Evolution

Which patterns appeared or disappeared over time:

- **"reply" disappears**: it drops from 63 → 50 → 20 → **0** → 1. This is the clearest disappearing pattern and suggests the comment structure changes from reply threads to more standalone comments later.
- **Topic terms grow**:
  - universe rises strongly (49 → 120 peak) and remains high.
  - time rises steadily to chunk 4 (14 → 40) and then falls slightly.
  - infinite rises strongly in chunks 3–4 (34 → 76 peak), then falls in chunk 5.
- **Late-appearing themes:** chunk 5 introduces terms like bang and hole, suggesting new topic directions (Big Bang / black hole).

These trends match the frequency-over-time line graph: topic-heavy words rise while *reply* collapses.

6.4 Correlation Matrix

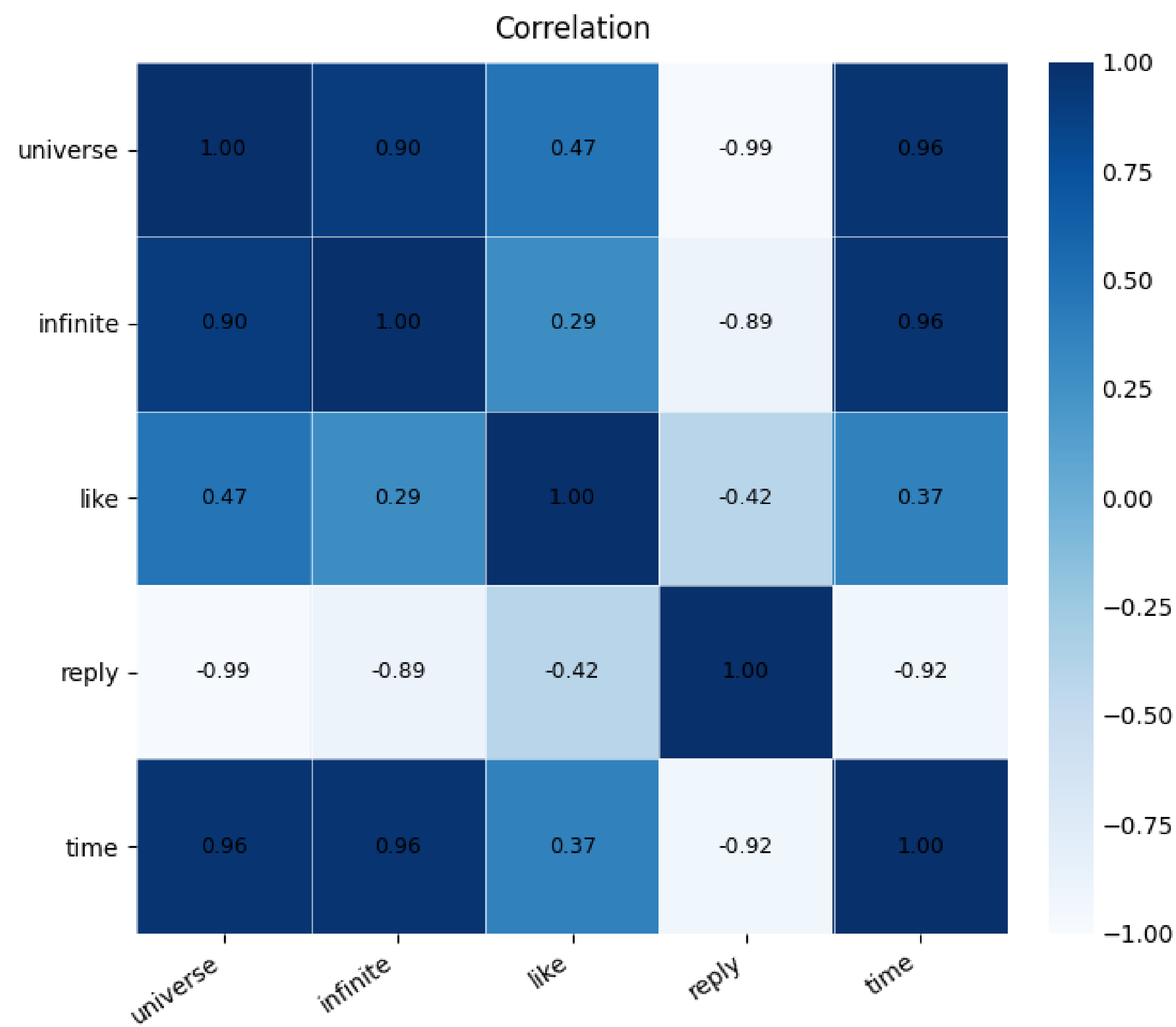

Figure 4: Correlation Matrix

How correlations reveal relationships between topics:

- Strong positive relationships (topic cluster):
    - universe ↔ time = **0.960937**
    - infinite ↔ time = **0.962259**
    - universe ↔ infinite = **0.904404**

These values show a consistent "topic cluster," meaning discussion about the universe tends to rise together with infinity and time. This fits the chunk outputs where the strongest recurring pair is (infinite, universe) and where time/space become more frequent later.

- Strong negative relationships (conversation vs topic shift):
    - reply ↔ universe = **-0.987999**
    - reply ↔ time = **-0.923537**
    - reply ↔ infinite = **-0.885582**

These strong negatives show that as topic-focused discussion rises, reply-style interaction collapses. This supports the interpretation that early chunks contain more thread/reply activity, while later chunks shift toward direct topic explanation and debate.

Insight:
The extreme negative correlation around reply suggests that the dataset is not only changing in topic words, but also changing in how people engage. Early comments are more interactive ("reply"), but later comments are more concept-driven (universe/infinite/time). This indicates a structural change in conversation style across the chunks.

6.5 Challenges Faced During Analysis

- **Token format handling:** cleaned_tokens can appear as strings in CSV, so type checking and safe conversion using ast.literal_eval() was necessary to avoid errors during counting.
- **Chunk splitting reliability:** splitting the DataFrame directly caused indexing issues in the environment, so splitting by index and slicing with .loc was used to keep each chunk as a proper DataFrame (which the lab requires).
- **Interpreting results beyond counting:** correlation values needed careful interpretation with support from trend graphs, instead of only reporting numbers.
- **Segmentation limitation:** chunking was performed by row order as a simulation of time; if the original dataset is not strictly chronological, this limits how confidently we can claim real time-based evolution. This limitation was acknowledged in this report.

7. Conclusion

This lab analyzed 739 YouTube comments in five chunks to simulate comments arriving over time. The discussion was mainly focused on "universe" and "infinite," with strong associations like (infinite, universe) appearing across chunks. Over time, topic words such as universe, infinite, and time increased, while reply dropped sharply and almost disappeared, suggesting a shift from reply-style interaction to more topic-focused comments. The correlation results supported this: universe/infinite/time moved together strongly, and reply moved in the opposite direction.